Math 118: Mathematical Methods of Data Theory

Lecture 4: QR Factorization and Least Squares

Instructor: Daniel Mckenzie

Dept. of Mathematics, UCLA

13 January 2020

Least Squares Again

The Problem: Find $\boldsymbol{x}^* = \underset{\boldsymbol{x}\in\mathbb{R}^n}{\arg\min} \|A\boldsymbol{x} - \boldsymbol{b}\|_2$ where $A \in \mathbb{R}^{m\times n}$ and $\boldsymbol{b} \in \mathbb{R}^m$ and $m \geq n$.

- Normal Equation: find $\boldsymbol{x}^*$ by solving $(A^\top A)\boldsymbol{x} = A^\top \boldsymbol{b}$.
- **Drawback:** $\kappa(A^\top A) = (\kappa(A))^2$. *Intuitively:* $A^\top A$ *is closer to singular than* A.
- **Example:** If $A = \begin{bmatrix} 1 & 1 \\ \epsilon & 0 \\ \epsilon & 0 \end{bmatrix}$ then $A^\top A = \begin{bmatrix} 1+\epsilon^2 & 1 \\ 1 & 1+\epsilon^2 \end{bmatrix}$
- QR decomposition provides an alternate approach which avoids this issue.

The QR decomposition for Least Squares

- Suppose can write $A = QR$ where $Q \in \mathbb{R}^{m \times m}$ is orthogonal and $R = \begin{bmatrix} R_1 \\ 0 \end{bmatrix} \in \mathbb{R}^{m \times n}$ with $R_1 \in \mathbb{R}^{n \times n}$ upper triangular.
- **Recall:**
  1. If $Q \in \mathbb{R}^{m \times m}$ orthogonal then so is $Q^\top = Q^{-1}$.
  2. $\|\boldsymbol{v}\|_2 = \|Q^\top \boldsymbol{v}\|_2$ for all vectors $\boldsymbol{v}$.
- $\|A\boldsymbol{x} - \boldsymbol{b}\|_2 = \|Q^\top (A\boldsymbol{x} - \boldsymbol{b})\| = \|Q^\top A\boldsymbol{x} - Q^\top \boldsymbol{b}\|_2$
  $= \|Q^\top (QR)\boldsymbol{x} - Q^\top \boldsymbol{b}\|_2 = \|Q^\top QR\boldsymbol{x} - Q^\top \boldsymbol{b}\|_2 = \|R\boldsymbol{x} - Q^\top \boldsymbol{b}\|_2$
- Solving $\arg\min \|R\boldsymbol{x} - Q^\top \boldsymbol{b}\|_2$ is easy:
  1. Write $\boldsymbol{b} = \begin{bmatrix} \boldsymbol{b}_1 \\ \boldsymbol{b}_2 \end{bmatrix}$ with $\boldsymbol{b}_1 \in \mathbb{R}^n$ and $\boldsymbol{b}_2 \in \mathbb{R}^{m-n}$
  2. $\|R\boldsymbol{x} - Q^\top \boldsymbol{b}\|_2 = \left\| \begin{bmatrix} R_1\boldsymbol{x} \\ \boldsymbol{0} \end{bmatrix} - \begin{bmatrix} Q^\top \boldsymbol{b}_1 \\ Q^\top \boldsymbol{b}_2 \end{bmatrix} \right\|_2 = \left\| \begin{bmatrix} R_1\boldsymbol{x} - Q^\top \boldsymbol{b}_1 \\ Q^\top \boldsymbol{b}_2 \end{bmatrix} \right\|_2$
  3. Minimize this by making $R_1\boldsymbol{x} = Q^\top \boldsymbol{b}_1$. Solve using backward substitution.

The QR decomposition

Theorem

Any $A \in \mathbb{R}^{m \times n}$ with $m \geq n$ can be written as $A = QR$ with $Q \in \mathbb{R}^{m \times m}$ orthogonal and $R = \begin{bmatrix} R_1 \\ 0 \end{bmatrix}$ with $R_1 \in \mathbb{R}^{n \times n}$ upper triangular.

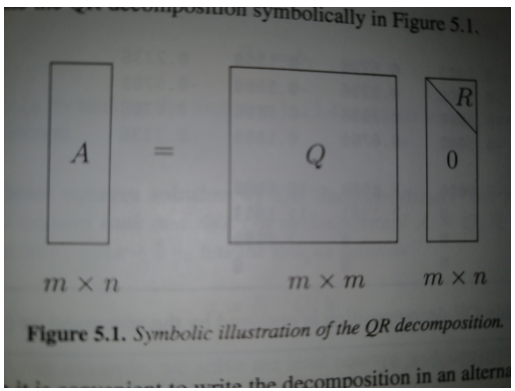


Figure 5.1. *Symbolic illustration of the QR decomposition.*

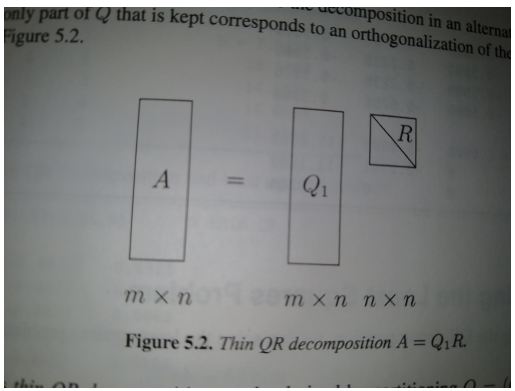


Figure 5.2. *Thin QR decomposition $A = Q_1 R$.*

Figure: Pictorial representation of QR decomposition (left) and thin QR decomposition (right). Note their R is our R_1.

- Write $Q = [Q_1, Q_2]$ with $Q_1 \in \mathbb{R}^{m \times n}$
- Observe: $A = QR = [Q_1, Q_2] \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = Q_1 R_1 + Q_2 0$
- Q_2 is redundant.
- **Thin QR:** just keep Q_1, R_1. Observe that $A = Q_1 R_1$ and $Q^\top \boldsymbol{b}_1 = Q_1^\top \boldsymbol{b}_1$.

Using the QR decomposition

- Exist good algorithms for finding QR decomposition. Keywords: Householder transformation and Given's rotation.
- Python, MATLAB C++ etc have built-in functions that find Q and R.
- Different methods for $\arg\min \|A\boldsymbol{x} - \boldsymbol{b}\|$:
  1. **Normal Equations:** Use for small m and n or theoretical analysis.
  2. QR **Decomposition:** Use most of the time.
  3. **Conjugate Gradient:** Use when A is sparse, when m, n very large or where speed is preferred to accuracy.

Eigenvalues and Diagonalization

Let $A \in \mathbb{R}^{n \times n}$ be a square matrix.

- **Recall:** λ is an **eigenvalue** and $\boldsymbol{v}$ is an **eigenvector**[1] if $A\boldsymbol{v} = \lambda \boldsymbol{v}$
- We say A is **diagonalizable** if we can write it as $A = V \Lambda V^{-1}$ where

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & \lambda_n \end{bmatrix} \text{ and } V = [\boldsymbol{v}_1, \boldsymbol{v}_2, \dots, \boldsymbol{v}_n].$$

- A is **unitarily diagonalizable** if $A = Q \Lambda Q^\top$ with Q orthogonal.
- Which matrices are diagonalizable?

Theorem

If $A^\top A = A A^\top$ we say A is **normal**. *A matrix is unitarily diagonalizable if and only if it is normal.*

- **Application:** Recall that $\|A\|_2 = \max_i |\lambda_i(A)|$ (for square A).
- **Question:** Can we extend this to non-square matrices?

[1] "eigen" means self in German. So an eigenvector is a vector which is mapped to itself.

Number of Operations

- FLOP = FLOating Point operation. Elementary operation on computer *eg* adding/multiplying two floating point numbers.
- "Algorithm $\mathcal{A}$ takes $O(n^2)$ operations." means $\mathcal{A}$ requires $\leq Cn^2$ FLOPS to solve problem of size n. Here C is a constant *independent of* n. [2]
- **Eg 1:** For $A \in \mathbb{R}^{n\times n}$ and $\boldsymbol{v} \in \mathbb{R}^n$ computing $A\boldsymbol{v}$ takes $O(??)$ operations.
- **Eg 2:** Tournament algorithm finds $\max_{i=1,\ldots,n} |v_i|$ in $O(n)$ operations.
- **Classically:** $O(n^2)$ algorithm always considered faster than $O(n^3)$.
- **Big Data + Stagnant growth in clock speed** $\Rightarrow$ this is no longer always the case.
- **Eg 1:** If algorithm is distributed then a **communication efficient** algorithm can be faster.
- **Eg 2:** If data set is v. large then a **memory efficient** algorithm can be faster.
- **Eg 3:** if data set is noisy then a fast approximate algorithm can be better than a slow exact algorithm.

[2] Note that C could be very large